

# Agent记忆框架怎么选？5大Agent Memory项目工程级横向对比

## How to Choose an Agent Memory Framework? Engineering-Level Comparison of 5 Agent Memory Projects

UP主: 唐国梁Tommy 时长: 15:11 播放: 32,149 点赞: 1,061

📖 技术教程与实操

### ⚡ 一句话总结 / TL;DR

系统对比5个开源Agent记忆框架的设计哲学与工程取舍，帮你建立记忆层设计的完整认知框架

A systematic comparison of 5 open-source Agent memory frameworks — design philosophies, engineering tradeoffs, and a complete mental model for memory layer design

### 🧠 我的解读

本视频是Agent记忆框架的横向评测系列上集，系统拆解了5个最具代表性的开源记忆框架：Text2Mem（操作语言层）、MemO（中间件层）、Letta（Agent系统层）、ReMe（用户透明层）、memU（主动Agent层）。UP主的核心论证不是'哪个最好'，而是展示记忆系统设计的5种不同范式——从定义操作语言、做中间件、做操作系统、做透明文件、到让记忆本身成为Agent。每个框架在不同的工程约束下做出了截然不同的设计选择，理解这些取舍比记住具体功能更重要。

### 记忆操作语言（Text2Mem） 📖 概念层

#### 📌 定义

为所有记忆系统定义一套通用的'操作指令集'，包含12个原子操作和五元JSON契约，是记忆层的ISA（指令集架构）

#### 📖 理论阐述

Text2Mem在LLM和存储之间插入一个中间表示层L2，把所有记忆操作收敛到12个原子操作，分三个阶段：ENC阶段（encode负责记忆诞生）、RET阶段（retrieve纯数据取回 + summarize让LLM生成摘要）、STO阶段（9个操作覆盖完整生命周期）。关键设计决策：1) retrieve和summarize分离——前者几ms返回，后者是一次完整LLM推理延迟上千倍，分开后可用不同超时/缓存/降级策略。2) 五元JSON契约（阶段、操作名、目标、参数、元数据）标准化所有操作。3) 双层验证：外层JSON Schema查格式，里层Pydantic查业务逻辑。4) 安全机制：dry\_run模拟执行 + confirmation显式确认二选一，防止LLM幻觉导致

灾难性操作。5) lock操作四种模式 (read\_only/no\_delete/append\_only/custom) + review机制实现RBAC风格权限控制。

#### 💡 生活类比

场景：就像CPU的指令集架构——你不需要为每款CPU重写编译器，只要ISA兼容就行

映射：ISA指令集 → 12个原子操作 / 编译器 → 记忆系统的具体实现 / CPU → 底层存储引擎 / 程序 → Agent的记忆需求

⚠️ 局限：类比无法解释的是：CPU ISA是硬件固定的，而Text2Mem的操作集仍在演进中，且目前只有SQLite参考实现

#### 🧠 个人洞察

Text2Mem的真正价值不是做一个能用的记忆系统，而是帮整个行业定义'记忆操作语言该长什么样'。这类似于SQL之于数据库的关系——先有标准语言，才能有繁荣的生态。当前短板（只有SQLite、无HTTP API、向量以JSON存text列）反而说明它在正确的抽象层做正确的事。

## 记忆中间件 (Mem0) 🔧 操作层

#### 📌 定义

当下Star最多的Agent记忆中间件，通过5个工厂模式、双存储并行、三种记忆类型实现开箱即用的记忆层

#### 📖 理论阐述

Mem0是三层架构：上层Memory API、中层LLM推理+向量检索逻辑层、下层存储层。工程成熟度体现在5个工厂模式：LLMFactory (17+提供商)、EmbeddingFactory (11+模型)、VectorStoreFactory (22+种存储)、GraphStoreFactory (4种图存储)、RerankerFactory (5种排序器)，用importlib动态加载未安装的包不报错。三种记忆类型：语义记忆（抽象事实）、情景记忆（具体事件）、程序记忆（Agent执行步骤，逐字保留用于崩溃恢复）。关键设计：1) UUID幻觉处理——LLM对长UUID处理差会幻觉，Mem0将其映射成简单整数让LLM只决定'对第3条执行update'。2) 双存储并行——向量存储负责语义相似搜索，图存储负责关系推理，用ThreadPoolExecutor并行跑结果合并。3) 双prompt策略——User memory extraction只看用户消息，Agent memory extraction只看助手消息，防止AI自我表达污染用户记忆。4) 多层级作用域隔离（用户/Agent/运行时三层）。

#### 💡 生活类比

场景：像一个瑞士军刀——什么功能都有，拿出来就能用，但每个功能都不是最强的

映射：瑞士军刀 → Mem0 / 各种工具 → 5个工厂 / 刀片/螺丝刀 → 不同的存储和LLM / 开箱即用 → 简单API调用

⚠️ 局限：类比无法解释成本瓶颈：每次add调用2-5次LLM，token随记忆规模线性增长，不适合高频实时写入

#### 🧠 个人洞察

Mem0的真正强项不是某一项黑科技，而是工程化的全家桶。它的设计哲学是'让80%的用户在30分钟内跑通记忆功能'。但这个哲学也意味着它不适合极致定制——如果你的场景有特殊的读写比例、延迟要求或存储约束，Mem0的抽象层可能反而成为瓶颈。

## 虚拟记忆操作系统 (Letta) 📖 概念层

#### 📌 定义

把操作系统虚拟内存思想完整搬进Agent架构，用Git版本化记忆 + SleepTime异步后台学习实现记忆自治

#### 📖 理论阐述

Letta的核心是三层记忆架构：Core Memory（核心内存，嵌在system prompt中每次推理可见，如CPU直接访问的RAM）、Archival Memory（归档内存，容量无限走向量检索，如磁盘）、Recall Memory（召回内存，存所有历史对话消息，如日志系统）。Core Memory的每个block是四元组（label路径命名空间/description用途/value内容/limit字符上限），limit默认10万字符是强制信息压缩约束。当上下文窗口快满时触发summarizer，驱逐30%消息写入recall memory，通过conversation search可再检索——真正意义上的虚拟内存无损分层。独特设计：1) Gate-enabled block manager引入真正gate管理记忆。2) 双存储——Git是真实来源，PG数据库是快速读缓存，每次记忆变更产生git commit（不可变/内容寻址/防静默损坏/完整历史可回溯/并发安全/可审计）。3) MemoryFS将记忆组织成目录树（人格设定/用户偏好/学到的技能），每份记忆是独立markdown文件。4) SleepTime agent——用户不交互时Agent持续自我改进，主agent只负责推理回复保持低延迟，每5步触发一次sleepTime agent用更大token预算做深度反思。

#### 💡 生活类比

场景：就像操作系统的虚拟内存——程序以为自己有无限内存，实际靠页面置换在RAM和磁盘间调度

映射：RAM → Core Memory / 磁盘 → Archival Memory / 页面置换 → summarizer驱逐机制 / 日志 → Recall Memory / 后台进程 → SleepTime agent

⚠️ 局限：认知成本高、数据库强依赖、80+依赖包——不是一个轻量级方案

#### 🧠 个人洞察

Letta是目前开源里记忆自治能力最强的系统。它的设计哲学是'让Agent像操作系统一样管理自己的记忆'。这与Mem0的'中间件'哲学截然不同——Mem0是给开发者用的API，Letta是给Agent自己用的OS。选择哪个取决于你想让'谁'来管理记忆：开发者还是Agent本身。

## 文件即记忆 (ReMe) 操作层

### 定义

阿里AgentScope出品，将记忆直接存为markdown文件，对用户完全透明可直接编辑，把控制权还给用户

### 理论阐述

ReMe走完全相反的哲学：传统做法记忆全存数据库是黑盒，ReMe把记忆直接存成markdown文件，用户打开就能看见、可以直接编辑、可以git版本控制。内部两套系统：ReMe Light（文件记忆，短期工作记忆）+ ReMe本体（向量记忆，长期语义记忆），时间维度错开。三个技术亮点：1) Delta File Watcher增量监控——检测文件是否纯追加模式，只处理新增部分节省92%的embedding API调用。2) When-to-use和content分离——向量嵌入建在when-to-use上而非content上，因为用户查询和when-to-use天然语义接近，大幅提升召回率。3) AI自主记忆管理——把文件操作工具给AI，让AI自己决定如何组织记忆。性能数据：千问38B + ReMe后综合得分超过没有记忆的千问34B。

### 生活类比

场景：像一个透明文件柜——你能看到所有文件、直接翻阅编辑，而不是把东西锁在保险箱里

映射：透明文件柜 → markdown文件 / 直接翻阅 → 用户可编辑 / 文件夹分类 → 记忆分类 / 保险箱 → 传统数据库存储

 **局限：**文件格式对大规模记忆的检索效率不如向量数据库，且用户手动编辑可能破坏记忆一致性

### 个人洞察

ReMe揭示了一个被其他框架忽视的价值：记忆的透明度和用户控制权。在AI越来越自主的趋势下，用户反而更想知道‘AI到底记住了我什么’。ReMe的‘文件即记忆’哲学不仅是一个技术选择，更是一种产品理念——信任用户，让用户参与记忆管理。

## 记忆即Agent (memU) 概念层

### 定义

范式最激进的项目——让记忆系统不再被动响应，而成为一个24/7后台主动运行的Agent

### 理论阐述

memU的核心洞察：前4个项目的记忆系统本质上都是被动的（用户说一句系统才add一条，用户问一句系统才search一次）。memU把关系反过来——记忆本身成为一个Agent。实现方式是双agent架构：Main Agent干老本行（听用户说话/调工具/生成回复），memU Bot只负责记忆（持续盯着每一次交互/后台提取整理分类）。代码实现很实在——用Python的asyncio.create\_task起异步后台任务，靠共享conversation\_messages列表通信。memU的文件系统是一个概念域：底层是数据库，文件夹对应category，文件对应memory item，符号链接对应交叉引用，挂载点对应resource。v1.4引入Silence-

Enhanced Memory（显著性感知记忆）：每条memory item带reinforcement\_count计数器，被检索一次加1，排序时根据强化次数加权——越常用的记忆越容易被再次召回，模拟人类'越经常想到的事越容易想起'的本质特性。OOMLO基准上跨所有推理任务平均准确率12.09%，与Mem0在同一基准上有竞争力。

#### 💡 生活类比

场景：像一个24小时值班的秘书——你不在的时候他也在整理文件、归档资料、预判你明天需要什么

映射：值班秘书 → memU Bot / 整理文件 → 记忆提取分类 / 预判需求 → 主动预测 / 你不在 → 用户未交互时

⚠️ 局限：双agent架构增加系统复杂度和token消耗，'主动预测'的准确率取决于场景

#### 🧠 个人洞察

memU最核心的转变是：从'Agent有记忆'到'记忆本身是一个Agent'。这不是语义游戏，而是架构模式的根本变化。当记忆系统有了自主性，它可以在用户不说话时也在学习、整理、优化——就像人类大脑的默认模式网络在休息时也在做信息整合。这可能是Agent记忆的终极形态，但也引入了最大的工程复杂度。

## 🔧 操作步骤

### 步骤 1：根据场景选择框架层

先判断你需要记忆系统的哪个抽象层：需要定义自己的记忆操作语言选Text2Mem，需要快速给产品加记忆层选Mem0，需要Agent自主管理记忆选Letta，需要记忆对用户透明可编辑选ReMe，需要记忆主动预测用户需求选memU

✅ 验证：明确自己的需求属于哪个层次，避免盲目选'最热门'的框架

⚠️ 避坑：不要因为Mem0 Star最多就选它——高频写入场景下它的成本瓶颈会成为致命问题

### 步骤 2：评估成本与复杂度

Mem0每次add调2-5次LLM，token随记忆规模线性增长，适合中低频写入。Letta依赖PostgreSQL和Git，80+依赖包，认知成本高。ReMe轻量但文件格式检索效率有限。memU双agent增加token消耗。Text2Mem目前只有SQLite参考实现

✅ 验证：了解每个框架的真实成本，而非只看功能列表

⚠️ 避坑：Demo里的性能数据（如Mem0准确率高26%、响应快91%）是在特定基准上的，不一定适用于你的业务场景

### 步骤 3：MVP原则逐步引入

从最简单最容易持续优化的形态开始，逐步引入记忆机制。每引入一个机制立即重新计算核心benchmark指标看是否实际提升。不要预设某种机制是最优的，Agent本质是随机的，几乎不存在放之四海而皆准的设计模式

✅ 验证：针对业务场景的垂类Agent逐渐收敛到最适合的记忆方案

⚠️ 避坑：照搬开源框架的架构而忽略自己业务的特殊读写比例、延迟要求和存储约束

## 🗨️ 思考题

Q1: 如果记忆系统有了自主性（如memU），如何防止它'过度记忆'导致信息过载？遗忘机制该怎么设计？

💡 提示：参考人类记忆的遗忘曲线和显著性筛选机制

Q2: Text2Mem定义操作语言的思路，是否可以类比为SQL之于数据库？如果是，Agent记忆的'SQL时刻'还差什么？

💡 提示：SQL的成功不仅在于语言本身，还在于查询优化器、事务机制、标准化生态

## 🎬 视频完整陈述

### 📋 视频结构

视频按5个框架逐一拆解：Text2Mem（操作语言层）→ Mem0（中间件层）→ Letta（Agent系统层）→ ReMe（用户透明层）→ memU（主动Agent层）。每个框架覆盖：核心理念、架构设计、关键亮点、工程取舍、适用场景。最后做总结对比，指出从'Agent有记忆'到'记忆本身是Agent'的范式转变。

### 🕒 0:00-2:10 开场：Agent记忆问题引入

UP主引入核心问题：大多数LLM Agent是无状态的——昨天聊完半小时的项目背景，今天打开新对话又把你当陌生人。要让Agent真正长期工作，记忆是绕不开的核心模块。本集目标：听完后对Agent记忆该怎么设计建立完整认知框架。涵盖5个框架：Text2Mem、Mem0、Letta、ReMe、memU。

### 🕒 2:10-6:20 Text2Mem——定义记忆操作语言

Text2Mem不是在做记忆系统，而是给所有记忆系统定义通用的操作语言。把所有记忆操作收敛到12个原子操作，分三个阶段：ENC（encode记忆诞生）、RET（retrieve纯取回 + summarize LLM摘要，两者延迟差上千倍所以分开设计）、STO（9个操作覆盖完整生命周期）。五元JSON契约（阶段/操作名/目标/参数/元数据）。安全机制：dry\_run模拟 + confirmation显式确认二选一防LLM幻觉。双层验证：外层JSON Schema + 里层Pydantic。lock操作四种模式（read\_only/no\_delete/append\_only/custom）+ review机制。短板：只有SQLite参考实现，无HTTP API。



## 🕒 6:20-9:20 Mem0——最流行的记忆中间件

VC孵化项目，GitHub几万Star。三层架构（API/逻辑/存储）。5个工厂模式（LLM 17+、Embedding 11+、VectorStore 22+、GraphStore 4、Reranker 5），importlib动态加载。三种记忆类型：语义/情景/程序（程序记忆逐字保留用于崩溃恢复）。UUID幻觉处理：映射成简单整数让LLM只决定操作。双存储并行（向量+图）。双prompt策略（User/Agent分离防污染）。多层作用域隔离。成本瓶颈：每次add调2-5次LLM，token随记忆规模线性增长，不适合高频实时写入。适合：AI对话助手、客服系统、医疗健康追踪。

## 🕒 9:20-12:40 Letta——虚拟记忆操作系统

原MemGPT，2024年9月更名Letta。把OS虚拟内存思想搬进Agent。三层记忆：Core Memory（嵌在system prompt，如RAM）、Archival Memory（向量检索，如磁盘）、Recall Memory（历史对话，如日志）。Core Memory每个block是四元组（label/description/value/limit），limit默认10万字符强制信息压缩。上下文满时summarizer驱逐30%消息写入recall memory——虚拟内存无损分层。Gate-enabled block manager。Git版本化记忆（不可变/内容寻址/并发安全/可审计）+ PG数据库快速读缓存。MemoryFS目录树组织记忆。Sleeptime agent用户不交互时持续自我改进。代价：认知成本高、数据库强依赖、80+依赖包。

## 🕒 12:40-14:00 ReMe——文件即记忆

阿里AgentScope出品。核心理念：传统记忆全存数据库是黑盒，ReMe直接存成markdown文件——用户打开就能看见、可以直接编辑、可以git版本控制。两套系统：ReMe Light（文件记忆/短期）+ ReMe本体（向量记忆/长期）。Delta File Watcher增量监控只处理新增部分节省92% API调用。When-to-use和content分离提升召回率。AI自主记忆管理。千问38B + ReMe后综合得分超过无记忆的千问34B——好的记忆系统可以让小模型打过大模型。

## 🕒 14:00-15:11 memU——记忆即Agent

范式最激进。前4个项目记忆都是被动的（用户说才add，问才search），memU把关系反过来——记忆本身成为Agent。双agent架构：Main Agent负责推理回复，memU Bot负责记忆（后台提取整理分类）。代码用asyncio.create\_task起异步任务。文件系统是概念域（数据库/文件夹/文件/符号链接/挂载点）。v1.4引入Silence-Enhanced Memory：每条记忆带reinforcement\_count计数器，被检索一次加1，排序时加权——越常用越容易想起，模拟人类记忆本质。OOMLO基准平均准确率12.09%，与Mem0有竞争力。

### 💬 UP主金句

「从agent有记忆到记忆本身是一个agent」「好的记忆系统可以让小模型打过大模型」「这种不信任LLM但能兜住LLM的设计哲学才是真正值得学的」

## 可信度评估

UP主有Agent开发实践经验，每个框架都深入到工程细节（而非只讲功能），既讲优势也讲代价和适用场景。但作为技术博主可能存在'技术乐观'倾向——对框架的局限性讲得不如优势详细。部分观众指出Mem0开源版功能可能被阉割，UP主讲的可能是专业版。

## 核心要点

- ▶ 5种记忆范式：操作语言层(Text2Mem)、中间件层(Mem0)、Agent系统层(Letta)、用户透明层(ReMe)、主动Agent层(memU)——不是选最好，是理解不同工程约束下的设计取舍
- ▶ Text2Mem的'不信任LLM但能兜住LLM'设计哲学值得学：dry\_run+confirmation双保险、双层验证、lock四模式——帮整个行业定义记忆操作语言
- ▶ Mem0是工程化全家桶但有真实成本瓶颈：每次add调2-5次LLM，token随记忆规模线性增长，不适合高频实时写入场景
- ▶ Letta用Git版本化记忆+Sleeptime agent实现记忆自治——是目前开源里记忆自治能力最强的系统，但80+依赖包认知成本高
- ▶ 从'Agent有记忆'到'记忆本身是Agent'——memU的范式转变揭示了记忆系统可能的终极形态

## 前置知识

需要了解LLM Agent的基本概念（上下文窗口、system prompt、工具调用）；了解向量数据库和embedding的基本原理；有实际使用或开发Agent的经验会更容易理解各框架的设计动机

## 难度评级

进阶——内容涉及多个框架的架构设计和工程取舍，需要Agent开发基础才能理解设计动机，但UP主的类比（CPU指令集/虚拟内存/瑞士军刀）降低了理解门槛

## 常见误解

1) Mem0 Star最多不代表最适合你的场景——高频写入场景下成本瓶颈会成为致命问题。2) '记忆框架'不等于'记忆方案'——每个框架在不同抽象层工作，可以组合使用。3) Demo基准数据（如Mem0准确率高26%）是在特定条件下测的，不直接等于你的业务效果。4) 有观众指出Mem0开源版功能可能被阉割，UP主讲的可能是专业版。



## 洞察与启发

这5个框架本质上代表了记忆系统设计的5种哲学：Text2Mem认为'先有标准语言才有生态'（SQL哲学）、Mem0认为'让80%的人快速跑通'（中间件哲学）、Letta认为'让Agent像OS一样自治'（操作系统哲学）、ReMe认为'把控制权还给用户'（透明哲学）、memU认为'记忆应该有自主性'（Agent哲学）。没有哪个哲学绝对正确，关键是在你的工程约束下哪个取舍最合理。更深一层：从被动记忆到主动记忆的转变，可能预示着Agent架构的下一个范式——不再是你'给'Agent记忆，而是Agent自己'有'记忆。

## 工具与资源清单

Text2Mem：定义记忆操作语言（12个原子操作+五元JSON契约）

△ 目前只有SQLite参考实现，无HTTP API

Mem0：开箱即用的记忆中间件（5工厂/双存储/三种记忆类型）

△ 每次add调2-5次LLM，成本随记忆规模增长

Letta：Agent自治记忆系统（三层记忆/Git版本化/Sleeptime）

△ 80+依赖包，需要PostgreSQL和Git

ReMe：用户透明的记忆系统（文件即记忆）

△ 文件格式检索效率有限

memU：主动Agent记忆系统（24/7后台记忆Agent）

△ 双agent架构增加复杂度和token消耗

## 避坑指南

- ▶ 不要因为Star数多就选Mem0——高频写入场景成本会爆
- ▶ Letta的Git版本化很酷但80+依赖包是真实的维护负担
- ▶ ReMe的markdown文件方案在大规模记忆下检索效率是瓶颈
- ▶ memU的'主动预测'在简单场景可能overkill
- ▶ Text2Mem的参考实现只有SQLite，生产环境需要自己扩展
- ▶ 所有框架的基准测试数据都是特定条件下的，不直接等于业务效果

## 后续行动

['根据自己的场景（高频写入？需要Agent自治？需要用户透明？）选择1-2个框架深入研究源码', '关注下集内容：MemOS/OpenViking/Hindsight/Second Me/MetaMem——更偏系统级和前沿创新', '用MVP原则从最简单的记忆形态开始，逐步引入机制并benchmark验证效果']

## 理解度测验

### 第1题 ★★

Text2Mem为什么把retrieve和summarize分成两个操作？

- A. 为了代码更简洁
- B. 因为两者延迟差异上千倍，分开后可用不同的超时/缓存/降级策略
- C. 因为summarize不需要LLM
- D. 为了兼容更多存储后端

答案：B

### 第2题 ★★

Letta的Core Memory默认字符上限是多少？这个设计的目的是什么？

- A. 1万字符，节省存储空间
- B. 10万字符，强制信息压缩约束，逼迫Agent主动做信息蒸馏
- C. 100万字符，保证信息完整性
- D. 无上限，让Agent自由管理

答案：B

### 第3题 ★

memU的Silence-Enhanced Memory机制模拟的是人类记忆的什么特性？

- A. 短期记忆容量限制
- B. 遗忘曲线
- C. 越经常想到的事情越容易想起（强化记忆）
- D. 情景记忆的时空关联

答案：C

#### 第4题 ★

以下哪个框架采用了'文件即记忆'的哲学，把记忆存为markdown文件？

- A. Text2Mem
- B. Mem0
- C. Letta
- D. ReMe

答案：D

#### 第5题 ★★

Mem0的双prompt策略（User/Agent extraction分离）的核心目的是什么？

- A. 减少token消耗
- B. 防止AI助手的自我表达污染用户记忆
- C. 提高检索速度
- D. 支持多语言

答案：B

### 💬 精选评论 / Top Comments

家电狗 18 赞

记忆系统确实是个大坑。多年以后，当我回忆往事，依然会想起那个一时冲动准备手搓一套记忆系统的夜晚，还有我那燃烧着的十几亿token

jojsjojdsdf 10 赞

短期记忆用向量数据库，只存最近100天的；超过100天的用GraphRAG抽象实体压缩掉具体细节，这就是目前最优的解决方案

BTSD96999 8 赞

记忆系统还是要学人，要有遗忘，被强化的内容才是值得记忆的

LiZeC 9 赞

本质问题是在调用模型时包含必要的信息，也就是一个检索问题。这些方案很多在解决存储和表示问题，反而不是问题。怎么把需要的数据捞出来才重要

bili\_9459439543 7 赞

Agent本质是随机的，几乎不存在放之四海而皆准的设计模式。最优实践是MVP原则加上benchmark打榜

## ✓ 推荐理由

如果你正在做或计划做Agent开发，记忆层是必须面对的核心模块。这个视频不推销某个框架，而是帮你建立'记忆系统设计'的完整认知地图——看完后你知道有哪些选择、每个选择的代价是什么、该怎么根据自己的场景做取舍。系列下集还有5个更前沿的项目。

## ✓ Recommendation

If you're building or planning to build Agents, the memory layer is a core module you must face. This video doesn't promote any framework — it helps you build a complete cognitive map of 'memory system design'. After watching, you'll know what options exist, what each costs, and how to make tradeoffs for your scenario. The next episode covers 5 more frontier projects.

 收藏夹考古: 已消化 4 个视频